SolidityScan | Powered by CRED SHIELDS

Security Assessment Report

XOX Faucet

26 March 2026

This security assessment report was prepared by SolidityScan.com, a cloud-based Smart Contract Scanner.

AICPA SOC | SCS Advocates OWASP OWASP Smart Contract Top 10 Security Risks & Vulnerabilities

Table of Contents

01 Vulnerability Classification and Severity

02 Executive Summary

03 Findings Summary

04 Vulnerability Details

BLOCK.NUMBER INCONSISTENCIES ON VARIOUS L2S

PRECISION LOSS DURING DIVISION BY LARGE NUMBERS

EOA-ONLY RESTRICTION CAN BE BYPASSED VIA CONTRACT CONSTRUCTION

PREDICTABLE PSEUDO-RANDOMNESS ENABLES PAYOUT MANIPULATION

OUTDATED COMPILER VERSION

ABI.ENCODEPACKED MAY CAUSE COLLISION

ADDING A RETURN STATEMENT WHEN THE FUNCTION DEFINES A NAMED RETURN VARIABLE IS REDUNDANT

BLOCK VALUES AS A PROXY FOR TIME

CONSIDER USING UINT48 FOR TIME-RELATED VARIABLES

CONTRACT NAME SHOULD USE PASCALCASE

MISSING @AUTHOR IN NATSPEC COMMENTS FOR CONTRACT DECLARATION

MISSING @DEV IN NATSPEC COMMENTS FOR CONTRACT DECLARATION

MISSING @DEV IN NATSPEC COMMENTS FOR FUNCTIONS

MISSING INDEXED KEYWORDS IN EVENTS

MISSING @INHERITDOC ON OVERRIDE FUNCTIONS

MISSING NATSPEC COMMENTS IN SCOPE BLOCKS

MISSING @NOTICE IN NATSPEC COMMENTS FOR CONSTRUCTORS

MISSING @NOTICE IN NATSPEC COMMENTS FOR FUNCTIONS

MISSING PAYABLE IN CALL FUNCTION

NAME MAPPING PARAMETERS
REVERT STATEMENTS WITHIN EXTERNAL AND PUBLIC FUNCTIONS CAN BE USED TO PERFORM DOS ATTACKS
UNUSED RECEIVE FALLBACK
AVOID RE-STORING VALUES
AVOID ZERO-TO-ONE STORAGE WRITES
CHEAPER INEQUALITIES IN IF()
DEFINE CONSTRUCTOR AS PAYABLE
FUNCTIONS CAN BE IN-LINED
REVERTING FUNCTIONS CAN BE PAYABLE
FUNCTION SHOULD RETURN STRUCT
GAS INEFFICIENCY DUE TO MULTIPLE OPERANDS IN SINGLE IF/ELSEIF CONDITION
OPTIMIZING ADDRESS ID MAPPING
PUBLIC CONSTANTS CAN BE PRIVATE
SMALLER DATA TYPES COST MORE
STORAGE VARIABLE CACHING IN MEMORY
SUPERFLUOUS EVENT FIELDS
UNNECESSARY DEFAULT VALUE INITIALIZATION
USING BITMAPS INSTEAD OF BOOLEANS FOR GAS OPTIMIZATION

05 Scan History

06 Disclaimer

01. Vulnerability Classification and Severity

Description

To enhance navigability, the document is organized in descending order of severity for easy reference. Issues are categorized as *Fixed*, *Pending Fix*, or *Won't Fix*, indicating their current status. *Won't Fix* denotes that the team is aware of the issue but has chosen not to resolve it. Issues labeled as *Pending Fix* state that the bug is yet to be resolved. Additionally, each issue's severity is assessed based on the risk of exploitation or the potential for other unexpected or unsafe behavior.

Critical

The issue affects the contract in such a way that funds may be lost, allocated incorrectly, or otherwise result in a significant loss.

High

High-severity vulnerabilities pose a significant risk to both the Smart Contract and the organization. They can lead to user fund losses, may have conditional requirements, and are challenging to exploit.

Medium

The issue affects the ability of the contract to operate in a way that doesn't significantly hinder its behavior.

Low

The issue has minimal impact on the contract's ability to operate.

Informational

The issue does not affect the contract's operational capability but is considered good practice to address.

Gas

This category deals with optimizing code and refactoring to conserve gas.

02. Executive Summary

XOX Faucet

Uploaded Solidity File(s)

Language	Audit Methodology	Website
Solidity	**Static Scanning**	-
Publishers/Owner Name	Organization	Contact Email
-	-	-

Security Score is GREAT

The SolidityScan score is calculated based on lines of code and weights assigned to each issue depending on the severity and confidence. To improve your score, view the detailed result and leverage the remediation solutions provided.

This report has been prepared for XOX Faucet using SolidityScan to scan and discover vulnerabilities and safe coding practices in their smart contract including the libraries used by the contract that are not officially recognized. The SolidityScan tool runs a comprehensive static analysis on the Solidity code and finds vulnerabilities ranging from minor gas optimizations to major vulnerabilities leading to the loss of funds. The coverage scope pays attention to all the informational and critical vulnerabilities with over 700+ modules. The scanning and auditing process covers the following areas:

Various common and uncommon attack vectors will be investigated to ensure that the smart contracts are secure from malicious actors. The scanner modules find and flag issues related to Gas optimizations that help in reducing the overall Gas cost It scans and evaluates the codebase against industry best practices and standards to ensure compliance It makes sure that the officially recognized libraries used in the code are secure and up to date.

The SolidityScan Team recommends running regular audit scans to identify any vulnerabilities that are introduced after XOX Faucet introduces new features or refactors the code.

03. Findings Summary

XOX Faucet
File Scan

Security Score	Scan duration	Lines of code
92.38/100	41 secs	374

0	0	2	3	62	60
Crit	High	Med	Low	Info	Gas

This audit report has not been verified by the SolidityScan team. To learn more about our published reports. click here

ACTION TAKEN

0	0	0	128
Fixed	*False Positive*	*Won't Fix*	*Pending Fix*

S. No.	Severity	Bug Type	Instances	Detection Method	Status
M001	Medium	BLOCK.NUMBER INCONSISTENCIES ON VARIOUS L2S	1	Automated	*Pending Fix*
M002	Medium	PRECISION LOSS DURING DIVISION BY LARGE NUMBERS	1	Automated	*Pending Fix*
L001	Low	EOA-ONLY RESTRICTION CAN BE BYPASSED VIA CONTRACT CONSTRUCTION	1	SolidityScan AI	*Pending Fix*
L002	Low	PREDICTABLE PSEUDO-RANDOMNESS ENABLES PAYOUT MANIPULATION	1	SolidityScan AI	*Pending Fix*
L003	Low	OUTDATED COMPILER VERSION	1	Automated	*Pending Fix*
I001	Informational	ABI.ENCODEPACKED MAY CAUSE COLLISION	1	Automated	*Pending Fix*
I002	Informational	ADDING A RETURN STATEMENT WHEN THE FUNCTION DEFINES A NAMED RETURN VARIABLE IS REDUNDANT	1	Automated	*Pending Fix*
I003	Informational	BLOCK VALUES AS A PROXY FOR TIME	7	Automated	*Pending Fix*
I004	Informational	CONSIDER USING UINT48 FOR TIME-RELATED VARIABLES	1	Automated	*Pending Fix*
I005	Informational	CONTRACT NAME SHOULD USE PASCALCASE	1	Automated	*Pending Fix*
I006	Informational	MISSING @AUTHOR IN NATSPEC COMMENTS FOR CONTRACT DECLARATION	1	Automated	*Pending Fix*
I007	Informational	MISSING @DEV IN NATSPEC COMMENTS FOR CONTRACT DECLARATION	1	Automated	*Pending Fix*

S. No.	Severity	Bug Type	Instances	Detection Method	Status
I008	Informational	MISSING @DEV IN NATSPEC COMMENTS FOR FUNCTIONS	10	Automated	*Pending Fix*
I009	Informational	MISSING INDEXED KEYWORDS IN EVENTS	1	Automated	*Pending Fix*
I010	Informational	MISSING @INHERITDOC ON OVERRIDE FUNCTIONS	3	Automated	*Pending Fix*
I011	Informational	MISSING NATSPEC COMMENTS IN SCOPE BLOCKS	9	Automated	*Pending Fix*
I012	Informational	MISSING @NOTICE IN NATSPEC COMMENTS FOR CONSTRUCTORS	1	Automated	*Pending Fix*
I013	Informational	MISSING @NOTICE IN NATSPEC COMMENTS FOR FUNCTIONS	2	Automated	*Pending Fix*
I014	Informational	MISSING PAYABLE IN CALL FUNCTION	1	Automated	*Pending Fix*
I015	Informational	NAME MAPPING PARAMETERS	3	Automated	*Pending Fix*
I016	Informational	REVERT STATEMENTS WITHIN EXTERNAL AND PUBLIC FUNCTIONS CAN BE USED TO PERFORM DOS ATTACKS	20	Automated	*Pending Fix*
I017	Informational	UNUSED RECEIVE FALLBACK	1	Automated	*Pending Fix*
G001	Gas	AVOID RE-STORING VALUES	5	Automated	*Pending Fix*
G002	Gas	AVOID ZERO-TO-ONE STORAGE WRITES	4	Automated	*Pending Fix*
G003	Gas	CHEAPER INEQUALITIES IN IF()	9	Automated	*Pending Fix*
G004	Gas	DEFINE CONSTRUCTOR AS PAYABLE	1	Automated	*Pending Fix*
G005	Gas	FUNCTIONS CAN BE IN-LINED	1	Automated	*Pending Fix*
G006	Gas	REVERTING FUNCTIONS CAN BE PAYABLE	9	Automated	*Pending Fix*
G007	Gas	FUNCTION SHOULD RETURN STRUCT	1	Automated	*Pending Fix*

S. No.	Severity	Bug Type	Instances	Detection Method	Status
G008	Gas	GAS INEFFICIENCY DUE TO MULTIPLE OPERANDS IN SINGLE IF/ELSEIF CONDITION	4	Automated	*Pending Fix*
G009	Gas	OPTIMIZING ADDRESS ID MAPPING	3	Automated	*Pending Fix*
G010	Gas	PUBLIC CONSTANTS CAN BE PRIVATE	13	Automated	*Pending Fix*
G011	Gas	SMALLER DATA TYPES COST MORE	3	Automated	*Pending Fix*
G012	Gas	STORAGE VARIABLE CACHING IN MEMORY	6	Automated	*Pending Fix*
G013	Gas	SUPERFLUOUS EVENT FIELDS	1	Automated	*Pending Fix*
G014	Gas	UNNECESSARY DEFAULT VALUE INITIALIZATION	1	Automated	*Pending Fix*
G015	Gas	USING BITMAPS INSTEAD OF BOOLEANS FOR GAS OPTIMIZATION	1	Automated	*Pending Fix*

04. Vulnerability Details

Issue Type

BLOCK.NUMBER INCONSISTENCIES ON VARIOUS L2S

S. No.	Severity	Detection Method	Instances
M001	Medium	Automated	1

Description

On Optimism, `block.number` represents the L2 block number, whereas on Arbitrum, it reflects the L1 block number, necessitating the use of `ArbSys(address(100)).arbBlockNumber()` on Arbitrum.
Caution is advised as L2 block numbers may occur more frequently than L1 block numbers, leading to timing inconsis tencies, especially in voting scenarios across multiple chains. Notably, OZ has addressed this concern in version 4.9 by modifying their governor code to use a clock instead of block numbers.

Bug ID	File Location	Line No.	Action Taken
SSP_126426_39	XOX Faucet.sol	L425 - L425	*Pending Fix*

Issue Type

PRECISION LOSS DURING DIVISION BY LARGE NUMBERS

S. No.	Severity	Detection Method	Instances
M002	Medium	Automated	1

Description

In Solidity, when dividing large numbers, precision loss can occur due to limitations in the Ethereum Virtual Machine (EVM). Solidity lacks native support for decimal or fractional numbers, leading to truncation of division results to integers. This can result in inaccuracies or unexpected behaviors, especially when the numerator is not significantly larger than the denominator.

Bug ID	File Location	Line No.	Action Taken
SSP_126426_46	XOX Faucet.sol	L209 - L209	*Pending Fix*

Issue Type

EOA-ONLY RESTRICTION CAN BE BYPASSED VIA CONTRACT CONSTRUCTION

S. No.	Severity	Detection Method	Instances
L001	Low	SolidityScan AI	1

Description

The EOA restriction relies on user.code.length == 0. During contract construction, code length is zero, allowing a contract to call claim() and bypass the onlyEOA guard. This allows automated or contract-based claim farming even when onlyEOA is enabled.

Bug ID	File Location	Line No.	Action Taken
SSP_126426_128	XOX Faucet.sol	L189 - L230	*Pending Fix*

Issue Type

PREDICTABLE PSEUDO-RANDOMNESS ENABLES PAYOUT MANIPULATION

S. No.	Severity	Detection Method	Instances
L002	Low	SolidityScan AI	1

Description

The payout amount is derived from _randomTokenCount(), which uses block.timestamp, blockhash(block.number-1), user address, and last claim time. These values are either known to the caller at call time or can be influenced by miners/validators. As a result, callers can time their claims (or miners can slightly adjust timestamps) to increase the probability of receiving the maximum payout within the configured range.

Bug ID	File Location	Line No.	Action Taken
SSP_126426_127	XOX Faucet.sol	L417 - L432	*Pending Fix*

Issue Type

OUTDATED COMPILER VERSION

S. No.	Severity	Detection Method	Instances
L003	Low	Automated	1

Description

Using an outdated compiler version can be problematic especially if there are publicly disclosed bugs and issues that affect the current compiler version.

Bug ID	File Location	Line No.	Action Taken
SSP_126426_29	XOX Faucet.sol	L2 - L2	*Pending Fix*

Issue Type

ABI.ENCODEPACKED MAY CAUSE COLLISION

S. No.	Severity	Detection Method	Instances
I001	Informational	Automated	1

Description

The contract is found to be using `abi.encodePacked()`, which may lead to hash collisions when handling multiple variable-length arguments. Hash collisions occur because `abi.encodePacked()` concatenates all elements in order without distinguishing between different data types or their lengths. This issue is particularly concerning when the concatenated data is controlled or influenced by external users, but it can also arise internally due to repetitive patterns or complex data structures.

Bug ID	File Location	Line No.	Action Taken
SSP_126426_38	XOX Faucet.sol	L422 - L422	⚠️ *Pending Fix*

Issue Type

ADDING A RETURN STATEMENT WHEN THE FUNCTION DEFINES A NAMED RETURN VARIABLE IS REDUNDANT

S. No.	Severity	Detection Method	Instances
I002	Informational	Automated	1

Description

The contract defines a named return despite having a return statement. This causes redundancy and can lead to pot ential confusion in the code.

Bug ID	File Location	Line No.	Action Taken
SSP_126426_100	XOX Faucet.sol	L244 - L295	*Pending Fix*

Issue Type

BLOCK VALUES AS A PROXY FOR TIME

S. No.	Severity	Detection Method	Instances
I003	Informational	Automated	7

Description

Contracts often need access to time values to perform certain types of functionality. Values such as `block.timestamp` and `block.number` can be used to determine the current time or the time delta. However, they are not recommended for most use cases.

For `block.number`, as Ethereum block times are generally around 14 seconds, the delta between blocks can be predicted. The block times, on the other hand, do not remain constant and are subject to change for a number of reasons, e.g., fork reorganizations and the difficulty bomb.

Due to variable block times, `block.number` should not be relied on for precise calculations of time.

Bug ID	File Location	Line No.	Action Taken
SSP_126426_48	XOX Faucet.sol	L199 - L199	*Pending Fix*
SSP_126426_49	XOX Faucet.sol	L217 - L217	*Pending Fix*
SSP_126426_50	XOX Faucet.sol	L275 - L275	*Pending Fix*
SSP_126426_51	XOX Faucet.sol	L303 - L303	*Pending Fix*

Bug ID	File Location	Line No.	Action Taken
SSP_126426_52	XOX Faucet.sol	L304 - L304	⚠️ *Pending Fix*
SSP_126426_53	XOX Faucet.sol	L424 - L424	⚠️ *Pending Fix*
SSP_126426_54	XOX Faucet.sol	L425 - L425	⚠️ *Pending Fix*

Issue Type

CONSIDER USING UINT48 FOR TIME-RELATED VARIABLES

S. No.	Severity	Detection Method	Instances
I004	Informational	Automated	1

Description

The contract defines time-related variables as `uint256`. Since Unix timestamps can be adequately represented usin g `uint48`, utilizing `uint256` for these variables results in unnecessary gas consumption due to larger storage require ments.

Bug ID	File Location	Line No.	Action Taken
SSP_126426_117	XOX Faucet.sol	L60 - L60	*Pending Fix*

Issue Type

CONTRACT NAME SHOULD USE PASCALCASE

S. No.	Severity	Detection Method	Instances
I005	Informational	Automated	1

Description

In Solidity, it's advisable to follow the established style guide, which dictates that contract names should be in PascalCase and match their respective file names. This adherence to consistent naming conventions not only enhances code readability and maintainability but also facilitates smoother collaboration among developers. By aligning with these recommendations, you make your code more accessible to others and reduce the likelihood of naming conflicts, contributing to a well-organized and efficient coding environment.

Bug ID	File Location	Line No.	Action Taken
SSP_126426_28	XOX Faucet.sol	L25 - L25	*Pending Fix*

Issue Type

MISSING @AUTHOR IN NATSPEC COMMENTS FOR CONTRACT DECLARATION

S. No.	Severity	Detection Method	Instances
I006	Informational	Automated	1

Description

Natspec `@author` tags are missing from contract declarations in the code. This reduces code clarity and makes it difficult to determine the original author or organization responsible for the contract. The absence of an `@author` tag can lead to challenges in tracking ownership, verifying authenticity, and maintaining proper documentation.

Bug ID	File Location	Line No.	Action Taken
SSP_126426_32	XOX Faucet.sol	L25 - L447	*Pending Fix*

Issue Type

MISSING @DEV IN NATSPEC COMMENTS FOR CONTRACT DECLARATION

S. No.	Severity	Detection Method	Instances
I007	Informational	Automated	1

Description

Natspec `@dev` tags are missing from contract declarations in the code. This reduces code clarity and makes it difficult for developers to understand the contract's internal logic, implementation details, and potential caveats.

Bug ID	File Location	Line No.	Action Taken
SSP_126426_99	XOX Faucet.sol	L25 - L447	*Pending Fix*

Issue Type

MISSING @DEV IN NATSPEC COMMENTS FOR FUNCTIONS

S. No.	Severity	Detection Method	Instances
I008	Informational	Automated	10

Description

Natspec @dev comments are missing from a function declaration in the code. The @dev tag provides a detailed explanation of a function's purpose, assumptions, and behavior. Lack of @dev documentation can: reduce code maintainability, making it harder for developers to understand the function's implementation details.

Bug ID	File Location	Line No.	Action Taken
SSP_126426_55	XOX Faucet.sol	L189 - L230	*Pending Fix*
SSP_126426_56	XOX Faucet.sol	L301 - L305	*Pending Fix*
SSP_126426_57	XOX Faucet.sol	L310 - L312	*Pending Fix*
SSP_126426_58	XOX Faucet.sol	L321 - L325	*Pending Fix*
SSP_126426_59	XOX Faucet.sol	L341 - L344	*Pending Fix*
SSP_126426_60	XOX Faucet.sol	L349 - L352	*Pending Fix*
SSP_126426_61	XOX Faucet.sol	L357 - L361	*Pending Fix*
SSP_126426_62	XOX Faucet.sol	L376 - L381	*Pending Fix*

Bug ID	File Location	Line No.	Action Taken
SSP_126426_63	XOX Faucet.sol	L386 - L393	⚠️ *Pending Fix*
SSP_126426_64	XOX Faucet.sol	L446 - L446	⚠️ *Pending Fix*

Issue Type

MISSING INDEXED KEYWORDS IN EVENTS

S. No.	Severity	Detection Method	Instances
I009	Informational	Automated	1

Description

Events are essential for tracking off-chain data and when the event paraemters are indexed they can be used as filter options which will help getting only the specific data instead of all the logs.

Bug ID	File Location	Line No.	Action Taken
SSP_126426_68	XOX Faucet.sol	L149 - L149	*Pending Fix*

Issue Type

MISSING @INHERITDOC ON OVERRIDE FUNCTIONS

S. No.	Severity	Detection Method	Instances
I010	Informational	Automated	3

Description

Overridden functions in Solidity should include the `@inheritdoc` tag in their NatSpec comments to ensure documentation consistency and clarity. Failing to include `@inheritdoc` can lead to incomplete or missing documentation, making it difficult for developers and auditors to understand the function's intended behavior. This lack of clarity can increase maintenance overhead and potentially lead to misinterpretations of contract logic.

Bug ID	File Location	Line No.	Action Taken
SSP_126426_78	XOX Faucet.sol	L165 - L168	*Pending Fix*
SSP_126426_79	XOX Faucet.sol	L417 - L432	*Pending Fix*
SSP_126426_80	XOX Faucet.sol	L437 - L441	*Pending Fix*

Issue Type

MISSING NATSPEC COMMENTS IN SCOPE BLOCKS

S. No.	Severity	Detection Method	Instances
I011	Informational	Automated	9

Description

NatSpec comments are missing from an unnamed scope block. Scope blocks can group computations or temporary l ogic and should be documented to clarify purpose and expected behavior. Lack of NatSpec documentation can redu ce readability, making it hard to understand why the scope block exists.

Bug ID	File Location	Line No.	Action Taken
SSP_126426_108	XOX Faucet.sol	L207 - L210	*Pending Fix*
SSP_126426_109	XOX Faucet.sol	L256 - L258	*Pending Fix*
SSP_126426_110	XOX Faucet.sol	L260 - L262	*Pending Fix*
SSP_126426_111	XOX Faucet.sol	L264 - L266	*Pending Fix*
SSP_126426_112	XOX Faucet.sol	L268 - L270	*Pending Fix*
SSP_126426_113	XOX Faucet.sol	L275 - L277	*Pending Fix*
SSP_126426_114	XOX Faucet.sol	L279 - L281	*Pending Fix*
SSP_126426_115	XOX Faucet.sol	L286 - L288	*Pending Fix*
SSP_126426_116	XOX Faucet.sol	L290 - L292	*Pending Fix*

Issue Type

MISSING @NOTICE IN NATSPEC COMMENTS FOR CONSTRUCTORS

S. No.	Severity	Detection Method	Instances
I012	Informational	Automated	1

Description

Natspec `@notice` comments are missing from a constructor declaration in the code. The `@notice` tag provides an o verview of what the constructor does, helping users and developers quickly understand its purpose.

Bug ID	File Location	Line No.	Action Taken
SSP_126426_47	XOX Faucet.sol	L165 - L168	*Pending Fix*

Issue Type

MISSING @NOTICE IN NATSPEC COMMENTS FOR FUNCTIONS

S. No.	Severity	Detection Method	Instances
I013	Informational	Automated	2

Description

Natspec `@notice` comments are missing from a function declaration in the code. The `@notice` tag provides an over view of what the function does, helping users and developers quickly understand its purpose.

Bug ID	File Location	Line No.	Action Taken
SSP_126426_30	XOX Faucet.sol	L417 - L432	*Pending Fix*
SSP_126426_31	XOX Faucet.sol	L437 - L441	*Pending Fix*

Issue Type

MISSING PAYABLE IN CALL FUNCTION

S. No.	Severity	Detection Method	Instances
I014	Informational	Automated	1

Description

The contract is using a `.call()` method to make external calls along with passing some Ether as `msg.value`. Since the function is not marked as `payable`, the transaction might fail if the contract does not have ETH.

Bug ID	File Location	Line No.	Action Taken
SSP_126426_81	XOX Faucet.sol	L403 - L403	*Pending Fix*

Issue Type

NAME MAPPING PARAMETERS

S. No.	Severity	Detection Method	Instances
I015	Informational	Automated	3

Description

After Solidity 0.8.18, a feature was introduced to name mapping parameters. This helps in defining a purpose for each mapping and makes the code more descriptive.

Bug ID	File Location	Line No.	Action Taken
SSP_126426_122	XOX Faucet.sol	L127 - L127	*Pending Fix*
SSP_126426_123	XOX Faucet.sol	L130 - L130	*Pending Fix*
SSP_126426_124	XOX Faucet.sol	L133 - L133	*Pending Fix*

Issue Type

REVERT STATEMENTS WITHIN EXTERNAL AND PUBLIC FUNCTIONS CAN BE USED TO PERFORM DOS ATTACKS

S. No.	Severity	Detection Method	Instances
I016	Informational	Automated	20

Description

In public and external functions, improper use of `revert` can be exploited for Denial of Service (DoS) attacks. An attacker can intentionally trigger these 'revert' conditions, causing legitimate transactions to consistently fail. For example, if a function relies on specific conditions from user input or contract state, an attacker could manipulate these to continually force reverts, blocking the function's execution.

Bug ID	File Location	Line No.	Action Taken
SSP_126426_1	XOX Faucet.sol	L192 - L192	*Pending Fix*
SSP_126426_2	XOX Faucet.sol	L193 - L193	*Pending Fix*
SSP_126426_3	XOX Faucet.sol	L194 - L194	*Pending Fix*
SSP_126426_4	XOX Faucet.sol	L195 - L195	*Pending Fix*
SSP_126426_5	XOX Faucet.sol	L199 - L199	*Pending Fix*

Bug ID	File Location	Line No.	Action Taken
SSP_126426_6	XOX Faucet.sol	L202 - L202	⚠️ *Pending Fix*
SSP_126426_7	XOX Faucet.sol	L212 - L212	⚠️ *Pending Fix*
SSP_126426_8	XOX Faucet.sol	L213 - L213	⚠️ *Pending Fix*
SSP_126426_9	XOX Faucet.sol	L322 - L322	⚠️ *Pending Fix*
SSP_126426_10	XOX Faucet.sol	L332 - L332	⚠️ *Pending Fix*
SSP_126426_11	XOX Faucet.sol	L358 - L358	⚠️ *Pending Fix*
SSP_126426_12	XOX Faucet.sol	L368 - L368	⚠️ *Pending Fix*
SSP_126426_13	XOX Faucet.sol	L377 - L377	⚠️ *Pending Fix*
SSP_126426_13	XOX Faucet.sol	L400 - L400	⚠️ *Pending Fix*
SSP_126426_14	XOX Faucet.sol	L378 - L378	⚠️ *Pending Fix*
SSP_126426_15	XOX Faucet.sol	L387 - L387	⚠️ *Pending Fix*
SSP_126426_16	XOX Faucet.sol	L388 - L388	⚠️ *Pending Fix*
SSP_126426_17	XOX Faucet.sol	L389 - L389	⚠️ *Pending Fix*
SSP_126426_17	XOX Faucet.sol	L401 - L401	⚠️ *Pending Fix*
SSP_126426_18	XOX Faucet.sol	L404 - L404	⚠️ *Pending Fix*

Issue Type

UNUSED RECEIVE FALLBACK

S. No.	Severity	Detection Method	Instances
I017	Informational	Automated	1

Description

The contract was found to be defining an empty function.
It is not recommended to leave them empty unless there's a specific use case such as to receive Ether via an empty `receive()` function.

Bug ID	File Location	Line No.	Action Taken
SSP_126426_84	XOX Faucet.sol	L446 - L446	*Pending Fix*

Issue Type

AVOID RE-STORING VALUES

S. No.	Severity	Detection Method	Instances
G001	Gas	Automated	5

Description

The function is found to be allowing re-storing the value in the contract's state variable even when the old value is equal to the new value. This practice results in unnecessary gas consumption due to the `Gsreset` operation (2900 gas), which could be avoided. If the old value and the new value are the same, not updating the storage would avoid this cost and could instead incur a `Gcoldsload` (2100 gas) or a `Gwarmaccess` (100 gas), potentially saving gas.

Bug ID	File Location	Line No.	Action Taken
SSP_126426_40	XOX Faucet.sol	L321 - L325	*Pending Fix*
SSP_126426_41	XOX Faucet.sol	L331 - L336	*Pending Fix*
SSP_126426_42	XOX Faucet.sol	L341 - L344	*Pending Fix*
SSP_126426_43	XOX Faucet.sol	L349 - L352	*Pending Fix*
SSP_126426_44	XOX Faucet.sol	L357 - L361	*Pending Fix*

Issue Type

AVOID ZERO-TO-ONE STORAGE WRITES

S. No.	Severity	Detection Method	Instances
G002	Gas	Automated	4

Description

Writing a storage variable from zero to a non-zero value costs 22,100 gas (20,000 for the write and 2,100 for cold access), making it one of the most expensive operations. This is why patterns like OpenZeppelin's `ReentrancyGuard` use `1` and `2` instead of `0` and `1`—to avoid the high cost of zero-to-non-zero writes. Non-zero to non-zero updates cost only 5,000 gas.

Bug ID	File Location	Line No.	Action Taken
SSP_126426_101	XOX Faucet.sol	L323 - L323	*Pending Fix*
SSP_126426_102	XOX Faucet.sol	L333 - L333	*Pending Fix*
SSP_126426_103	XOX Faucet.sol	L334 - L334	*Pending Fix*
SSP_126426_104	XOX Faucet.sol	L342 - L342	*Pending Fix*

Issue Type

CHEAPER INEQUALITIES IN IF()

S. No.	Severity	Detection Method	Instances
G003	Gas	Automated	9

Description

The contract was found to be doing comparisons using inequalities inside the if statement.
When inside the `if` statements, non-strict inequalities `(>=, <=)` are usually cheaper than the strict equalities `(>, <)`.

Bug ID	File Location	Line No.	Action Taken
SSP_126426_69	XOX Faucet.sol	L195 - L195	*Pending Fix*
SSP_126426_70	XOX Faucet.sol	L199 - L199	*Pending Fix*
SSP_126426_71	XOX Faucet.sol	L207 - L207	*Pending Fix*
SSP_126426_72	XOX Faucet.sol	L213 - L213	*Pending Fix*
SSP_126426_73	XOX Faucet.sol	L268 - L268	*Pending Fix*
SSP_126426_74	XOX Faucet.sol	L275 - L275	*Pending Fix*
SSP_126426_75	XOX Faucet.sol	L290 - L290	*Pending Fix*
SSP_126426_76	XOX Faucet.sol	L322 - L322	*Pending Fix*

Bug ID	File Location	Line No.	Action Taken
SSP_126426_77	XOX Faucet.sol	L332 - L332	⚠️ *Pending Fix*

Issue Type

DEFINE CONSTRUCTOR AS PAYABLE

S. No.	Severity	Detection Method	Instances
G004	Gas	Automated	1

Description

Developers can save around 10 opcodes and some gas if the constructors are defined as payable.
However, it should be noted that it comes with risks because payable constructors can accept ETH during deployment.

Bug ID	File Location	Line No.	Action Taken
SSP_126426_37	XOX Faucet.sol	L165 - L168	*Pending Fix*

Issue Type

FUNCTIONS CAN BE IN-LINED

S. No.	Severity	Detection Method	Instances
G005	Gas	Automated	1

Description

The internal function was called only once throughout the contract. Internal functions cost more gas due to additiona l JUMP instructions and stack operations.

Bug ID	File Location	Line No.	Action Taken
SSP_126426_45	XOX Faucet.sol	L417 - L432	*Pending Fix*

Issue Type

REVERTING FUNCTIONS CAN BE PAYABLE

S. No.	Severity	Detection Method	Instances
G006	Gas	Automated	9

Description

If a function modifier such as `onlyOwner` is used, the function will revert if a normal user tries to pay the function. Marking the function as payable will lower the gas cost for legitimate callers because the compiler will not include checks for whether a payment was provided.

Bug ID	File Location	Line No.	Action Taken
SSP_126426_19	XOX Faucet.sol	L321 - L325	*Pending Fix*
SSP_126426_20	XOX Faucet.sol	L331 - L336	*Pending Fix*
SSP_126426_21	XOX Faucet.sol	L341 - L344	*Pending Fix*
SSP_126426_22	XOX Faucet.sol	L349 - L352	*Pending Fix*
SSP_126426_23	XOX Faucet.sol	L357 - L361	*Pending Fix*
SSP_126426_24	XOX Faucet.sol	L367 - L371	*Pending Fix*
SSP_126426_25	XOX Faucet.sol	L376 - L381	*Pending Fix*
SSP_126426_26	XOX Faucet.sol	L386 - L393	*Pending Fix*

Bug ID	File Location	Line No.	Action Taken
SSP_126426_27	XOX Faucet.sol	L399 - L407	⚠️ *Pending Fix*

Issue Type

FUNCTION SHOULD RETURN STRUCT

S. No.	Severity	Detection Method	Instances
G007	Gas	Automated	1

Description

The function was detected to be returning multiple values.
Consider using a `struct` instead of multiple return values for the function. It can improve code readability.

Bug ID	File Location	Line No.	Action Taken
SSP_126426_82	XOX Faucet.sol	L244 - L295	*Pending Fix*

Issue Type

GAS INEFFICIENCY DUE TO MULTIPLE OPERANDS IN SINGLE IF/ELSEIF CONDITION

S. No.	Severity	Detection Method	Instances
G008	Gas	Automated	4

Description

The contract is found to use multiple operands within a single `if` or `else if` statement, which can lead to unnecessary gas consumption due to the way the EVM evaluates compound boolean expressions. Each operand in a compound condition is evaluated even if the first condition fails, unless short-circuiting occurs, and the combined logic can result in more complex bytecode and higher gas usage compared to using nested `if` statements. This inefficiency is particularly relevant in functions that are called frequently or within loops.

Bug ID	File Location	Line No.	Action Taken
SSP_126426_118	XOX Faucet.sol	L194 - L194	*Pending Fix*
SSP_126426_119	XOX Faucet.sol	L264 - L266	*Pending Fix*
SSP_126426_120	XOX Faucet.sol	L322 - L322	*Pending Fix*
SSP_126426_121	XOX Faucet.sol	L332 - L332	*Pending Fix*

Issue Type

OPTIMIZING ADDRESS ID MAPPING

S. No.	Severity	Detection Method	Instances
G009	Gas	Automated	3

Description

Combining multiple address/ID mappings into a single mapping using a struct enhances storage efficiency, simplifies code, and reduces gas costs, resulting in a more streamlined and cost-effective smart contract design.
It saves storage slot for the mapping and depending on the circumstances and sizes of types, it can avoid a Gsset (2 0000 gas) per mapping combined. Reads and subsequent writes can also be cheaper when a function requires both values and they fit in the same storage slot.

Bug ID	File Location	Line No.	Action Taken
SSP_126426_65	XOX Faucet.sol	L127 - L127	*Pending Fix*
SSP_126426_66	XOX Faucet.sol	L130 - L130	*Pending Fix*
SSP_126426_67	XOX Faucet.sol	L133 - L133	*Pending Fix*

Issue Type

PUBLIC CONSTANTS CAN BE PRIVATE

S. No.	Severity	Detection Method	Instances
G010	Gas	Automated	13

Description

Public constant variables cost more gas because the EVM automatically creates getter functions for them and adds entries to the method ID table. The values can be read from the source code instead.

Bug ID	File Location	Line No.	Action Taken
SSP_126426_86	XOX Faucet.sol	L51 - L51	*Pending Fix*
SSP_126426_87	XOX Faucet.sol	L54 - L54	*Pending Fix*
SSP_126426_88	XOX Faucet.sol	L57 - L57	*Pending Fix*
SSP_126426_89	XOX Faucet.sol	L60 - L60	*Pending Fix*
SSP_126426_90	XOX Faucet.sol	L63 - L63	*Pending Fix*
SSP_126426_91	XOX Faucet.sol	L66 - L66	*Pending Fix*
SSP_126426_92	XOX Faucet.sol	L68 - L68	*Pending Fix*
SSP_126426_93	XOX Faucet.sol	L70 - L70	*Pending Fix*
SSP_126426_94	XOX Faucet.sol	L72 - L72	*Pending Fix*
SSP_126426_95	XOX Faucet.sol	L74 - L74	*Pending Fix*

Bug ID	File Location	Line No.	Action Taken
SSP_126426_96	XOX Faucet.sol	L76 - L76	⚠️ *Pending Fix*
SSP_126426_97	XOX Faucet.sol	L78 - L78	⚠️ *Pending Fix*
SSP_126426_98	XOX Faucet.sol	L80 - L80	⚠️ *Pending Fix*

Issue Type

SMALLER DATA TYPES COST MORE

S. No.	Severity	Detection Method	Instances
G011	Gas	Automated	3

Description

Usage of smaller integer types such as `uint8`, `uint16`, `int8`, or `int16` in arithmetic operations incur additional gas costs compared to the default `uint` and `int` types, which are typically `uint256` and `int256` respectively.

Bug ID	File Location	Line No.	Action Taken
SSP_126426_105	XOX Faucet.sol	L283 - L283	*Pending Fix*
SSP_126426_106	XOX Faucet.sol	L418 - L418	*Pending Fix*
SSP_126426_107	XOX Faucet.sol	L431 - L431	*Pending Fix*

Issue Type

STORAGE VARIABLE CACHING IN MEMORY

S. No.	Severity	Detection Method	Instances
G012	Gas	Automated	6

Description

The contract is using the state variable multiple times in the function.
SLOADs are expensive (100 gas after the 1st one) compared to MLOAD / MSTORE (3 gas each).

Bug ID	File Location	Line No.	Action Taken
SSP_126426_33	XOX Faucet.sol	L189 - L230	*Pending Fix*
SSP_126426_33	XOX Faucet.sol	L189 - L230	*Pending Fix*
SSP_126426_33	XOX Faucet.sol	L189 - L230	*Pending Fix*
SSP_126426_34	XOX Faucet.sol	L244 - L295	*Pending Fix*
SSP_126426_35	XOX Faucet.sol	L417 - L432	*Pending Fix*
SSP_126426_36	XOX Faucet.sol	L437 - L441	*Pending Fix*

Issue Type

SUPERFLUOUS EVENT FIELDS

S. No.	Severity	Detection Method	Instances
G013	Gas	Automated	1

Description

block.timestamp and block.number are by default added to event information. Adding them manually costs extra g as.

Bug ID	File Location	Line No.	Action Taken
SSP_126426_85	XOX Faucet.sol	L222 - L229	*Pending Fix*

Issue Type

UNNECESSARY DEFAULT VALUE INITIALIZATION

S. No.	Severity	Detection Method	Instances
G014	Gas	Automated	1

Description

The contract was found to be initializing the value of the variable to it's default value.
This is redundant and not required.

Bug ID	File Location	Line No.	Action Taken
SSP_126426_126	XOX Faucet.sol	L120 - L120	*Pending Fix*

Issue Type

USING BITMAPS INSTEAD OF BOOLEANS FOR GAS OPTIMIZATION

S. No.	Severity	Detection Method	Instances
G015	Gas	Automated	1

Description

In Solidity, each `bool` variable consumes 1 byte of storage, but an Ethereum storage slot is 32 bytes (256 bits). This means storing multiple boolean values separately leads to inefficient gas usage. A more optimized approach is to use bitmaps, which pack multiple boolean values into a single storage slot, significantly reducing gas costs.

Bug ID	File Location	Line No.	Action Taken
SSP_126426_83	XOX Faucet.sol	L25 - L25	*Pending Fix*

05. Scan History

● Critical ● High ● Medium ● Low ● Informational ● Gas

No	Date	Security Score	Scan Overview
1.	2026-03-26	**92.38**	● 0 ● 0 ● 2 ● 3 ● 62 ● 60

06. Disclaimer

The Reports neither endorse nor condemn any specific project or team, nor do they guarantee the security of any specific project. The contents of this report do not, and should not be interpreted as having any bearing on, the economics of tokens, token sales, or any other goods, services, or assets.

The security audit is not meant to replace functional testing done before a software release.

There is no warranty that all possible security issues of a particular smart contract(s) will be found by the tool, i.e., It is not guaranteed that there will not be any further findings based solely on the results of this evaluation.

Emerging technologies such as Smart Contracts and Solidity carry a high level of technical risk and uncertainty. There is no warranty or representation made by this report to any Third Party in regards to the quality of code, the business model or the proprietors of any such business model, or the legal compliance of any business.

In no way should a third party use these reports to make any decisions about buying or selling a token, product, service, or any other asset. It should be noted that this report is not investment advice, is not intended to be relied on as investment advice, and has no endorsement of this project or team. It does not serve as a guarantee as to the project's absolute security.

The assessment provided by SolidityScan is subject to dependencies and under continuing development. You agree that your access and/or use, including but not limited to any services, reports, and materials, will be at your sole risk on an as-is, where-is, and as-available basis. SolidityScan owes no duty to any third party by virtue of publishing these Reports.

As one audit-based assessment cannot be considered comprehensive, we always recommend proceeding with several independent manual audits including manual audit and a public bug bounty program to ensure the security of the smart contracts.